

کامپایلر NNGraph DSL

یک زبان دامنه‌محور برای توصیف معماری شبکه‌های عصبی
و تولید خودکار کد PyTorch

گزارش فنی پروژه

۲۶ خرداد ۱۴۰۵

فهرست مطالب

۳	چکیده
۴	۱ مقدمه
۴	۱.۱ انگیزه
۵	۲.۱ هدف
۵	۳.۱ دامنه کاربرد
۶	۲ کارهای مرتبط
۶	۱.۲ چارچوب NADER
۶	۲.۲ FANTLR
۶	۱.۲.۲ الگوی Listener در برابر Visitor
۷	۳.۲ پروژه ANTLR EVM
۸	۳ طراحی زبان
۸	۱.۳ ساختار برنامه
۹	۲.۳ گرامر
۱۰	۳.۳ سیستم انواع
۱۰	۴.۳ انواع لایه
۱۰	۱.۴.۳ لایه‌های پارامتردار
۱۱	۲.۴.۳ فعال‌سازی‌ها
۱۱	۳.۴.۳ عملیات گراف (بدون ماژول)
۱۲	۵.۳ نحو یال
۱۲	۶.۳ کامنت‌ها
۱۳	۴ پیاده‌سازی
۱۳	۱.۴ معماری کامپایلر
۱۴	۱.۱.۴ مسئولیت فایل‌ها
۱۴	۲.۴ تحلیل‌گر معنایی

۱۴	انتشار مقدار در درخت	۱.۲.۴
۱۵	قوانین اعتبارسنجی	۲.۲.۴
۱۵	قالب پیام خطا	۳.۲.۴
۱۵	FalseCtx برای خطاهای پس از پیمایش	۴.۲.۴
۱۶	مولد کد	۳.۴
۱۶	الگوریتم تخصیص متغیر	۱.۳.۴
۱۸	موارد خاص تولید کد	۲.۳.۴
۱۸	MultiHeadAttn در batch_first	۳.۳.۴
۱۸	ساختار کد خروجی	۴.۳.۴

۵ ارزیابی

۲۰	مجموعه آزمایش	۱.۵
۲۱	نمونه‌های کارکرده	۲.۵
۲۱	اثبات درستی متغیرها	۳.۵
۲۲	آمار کدبیس	۴.۵

۶ تصمیمات طراحی

۲۳	Listener در برابر Visitor	۱.۶
۲۳	load_from_listener() در برابر generate(traversal)	۲.۶
۲۳	نام‌گذاری متغیر با node_id	۳.۶

۷ نتیجه‌گیری و کارهای آینده

۲۴	نتیجه‌گیری	۱.۷
۲۴	کارهای آینده	۲.۷

منابع

۲۶

چکیده

NNGraph DSL یک زبان دامنه‌محور (Domain-Specific Language) است که برای توصیف معماری شبکه‌های عصبی به صورت گراف جهت‌دار طراحی شده است. فایل‌های `nng` را به عنوان ورودی دریافت کرده و کد `PyTorch` اجرایی تولید می‌کند که یک زیرکلاس `nn.Module` کامل با متدهای `__init__` و `forward` است.

کامپایلر با `ANTLR4` نسخه `۲.۱۳.۴` ساخته شده و از الگوی `Listener` برای تحلیل معنایی و الگوریتم مرتب‌سازی توپولوژیک `Kahn` هم در تحلیل‌گر معنایی (تشخیص دور) و هم در مولد کد (ترتیب اجرا) استفاده می‌کند.

مجموعه آزمایش شامل ۵۵ تست در چهار فایل است. چهار معماری نمونه—`MLP`، `ResBlock`، `InceptionBlock` و `TransformerEncoder`—با موفقیت کامپایل شده و `forward pass` آن‌ها با شکل خروجی صحیح اجرا شده است.

فصل ۱

مقدمه

۱.۱ انگیزه

نوشتن زیرکلاس‌های `nn.Module` در PyTorch به صورت دستی برای معماری‌های پیچیده، تکراری و مستعد خطاست. برای یک شبکه با ده لایه، برنامه‌نویس باید:

- در `__init__` هر لایه را جداگانه با پارامترهای صحیح تعریف کند
- در `forward` ترتیب اجرا را به صورت دستی مدیریت کند
- در توپولوژی‌های شاخه‌دار، متغیرهای میانی را نام‌گذاری کند
- مطمئن شود هیچ تانسور مشترکی پیش از استفاده بازنویسی نمی‌شود

مثال ۱.۱.۱ ► نمونه boilerplate دستی برای یک ResBlock

برنامه‌نویس باید این ساختار را برای یک ResBlock ساده بنویسد:

```
class ResBlock(nn.Module):
    def __init__(self):
        super(ResBlock, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=64, out_channels=64,
                                kernel_size=3, stride=1, padding=1)
        self.bn1 = nn.BatchNorm2d(num_features=64)
        self.relu1 = nn.ReLU()
        # ... □ □□ □□□□ □□□□□

    def forward(self, x):
```

```

identity = x
x = self.conv1(x)
x = self.bn1(x)
x = self.relu1(x)
# ... ۱۱۱۱۱۱ ۱۱۱۱ ۱۱۱۱۱ ۱۱۱۱۱۱۱۱۱
x = x + identity
return x

```

با NNGraph DSL، معماری یک‌بار به صورت گراف توصیف می‌شود و کامپایلر این کد را تولید می‌کند.

۲.۱ هدف

هدف NNGraph DSL این است که فایل `nng` تنها منبع حقیقت معماری باشد. برنامه‌نویس گراف را توصیف می‌کند؛ کامپایلر تمام boilerplate را تولید می‌کند.

۳.۱ دامنه کاربرد

این ابزار برای توصیف معماری‌های ثابت (نه پویا) مناسب است—شبکه‌هایی که گراف محاسباتی آن‌ها در زمان تعریف مدل مشخص است. پشتیبانی از کنترل جریان پویا (`else/if` درون `forward`) در نسخه‌های آینده برنامه‌ریزی شده است.

فصل ۲

کارهای مرتبط

۱.۲ چارچوب NADER

تعریف ۱.۱.۲ NADER

NADER (Neural Architecture Design via Representation) چارچوبی است که معماری شبکه‌های عصبی را به صورت گراف محاسباتی جهت‌دار (DAG) مدل می‌کند. هر گره یک عملیات لایه است و هر یال جریان تنسور را نشان می‌دهد.

NNGraph DSL این مدل گراف را به عنوان پایه طراحی زبان به کار می‌گیرد.

۲.۲ ANTLR

تعریف ۱.۲.۲ ANTLR

ANTLR (ANother Tool for Language Recognition) ابزاری است که از یک فایل گرامر g4 . به صورت خودکار lexer، parser و listener/visitor تولید می‌کند. نسخه مورد استفاده: ANTLR ۴.۱۳.۲.

۱.۲.۲ الگوی Listener در برابر Visitor

ANTLR^۴ دو الگو برای پیمایش درخت تجزیه ارائه می‌دهد:

• **Listener**: ParseTreeWalker به صورت خودکار درخت را پیمایش می‌کند و متدهای `exit*/enter*`

را فراخوانی می‌کند. هیچ مقدار بازگشتی نیاز نیست.

• **Visitor**: برنامه‌نویس پیمایش را کنترل می‌کند و هر متد یک مقدار برمی‌گرداند.

این کامپایلر از الگوی Listener استفاده می‌کند. متدهای *exit در custom_listener.py حالت را در self.nodes و self.edges انباشته می‌کنند—این رویکرد برای ساختن یک مدل گراف از یک درخت تجزیه طبیعی‌تر است.

۳.۲ پروژه ANTLR EVM

این کامپایلر سبک و ساختار کد را از پروژه EVM موجود در PycharmProjects/Antlr به ارث می‌برد. سه فایل از آن پروژه مستقیماً استفاده می‌شوند:

• ast.py – کلاس AST و TreeNode

• make_ast_subtree.py – ترکیب گره‌های درخت تجزیه

• ast_to_networkx_graph.py – تجسم اختیاری گراف

فصل ۳

طراحی زبان

۱.۳ ساختار برنامه

هر فایل nng . از سه بلوک تشکیل شده است. بلوک config اختیاری است:

```
model <Name> {
    input  <id> : tensor(<shape>)
    output <id>
}

graph {
    node <id> : <LayerType>(<params>)
    ...
    edge <src> -> <dst>
    edge <src> -> <dst> [label="<string>"]
    ...
}

config {
    batch_size = <int>
    device      = "cpu" | "cuda"
}
```

تعریف ۱.۱.۳ > بلوک model

بلوک model نام کلاس تولیدشده، شناسه گره ورودی و شکل تنسور ورودی، و شناسه گره خروجی را مشخص می‌کند.

تعریف ۲.۱.۳ > بلوک graph

بلوک graph تمام گره‌های لایه (node) و یال‌های جهت‌دار (edge) را تعریف می‌کند. ترتیب اعلان یال‌ها اهمیتی ندارد؛ کامپایلر ترتیب اجرا را از ساختار گراف محاسبه می‌کند.

تعریف ۳.۱.۳ > بلوک config

بلوک config مقادیری را برای بلوک __main__ کد تولیدشده تعیین می‌کند. batch_size پیش‌فرض ۱ و device پیش‌فرض 'cpu' است.

۲.۳ گرامر

تعریف ۱.۲.۳ > آمار گرامر NNGraph.g

- فایل گرامر: NNGraph.g4 (۵۶ خط)
- قوانین تجزیه‌گر: ۱۴ قانون (snake_case)
- توکن‌های واژه‌نگار: ۲۰ توکن (ALL_CAPS)

قوانین تجزیه‌گر:

```

program      : model_block graph_block config_block? EOF;
model_block  : MODEL ID '{' input_decl output_decl '}';
input_decl   : INPUT ID ':' TENSOR shape_expr;
output_decl  : OUTPUT ID;
graph_block  : GRAPH '{' (node_decl | edge_decl)* '}';
node_decl    : NODE ID ':' layer_expr;
layer_expr   : ID '(' param_list? ')';
param_list   : param (',' param)*;
param        : ID '=' value;
edge_decl    : EDGE ID ARROW ID ('[' LABEL '=' STRING ''])?;
config_block: CONFIG '{' config_entry* '}';
config_entry: ID '=' value;

```

```

value          : FLOAT_LITERAL | INT_LITERAL | BOOL_LITERAL
                | STRING | NONE | shape_expr;
shape_expr     : '(' INT_LITERAL (',' INT_LITERAL)* ')';

```

نکته مهم طراحی: `input_decl` از `shape_expr` TENSOR استفاده می‌کند نه TENSOR. زیرا `'(' shape_expr ')'`، خود پرانتزها را در بر می‌گیرد. این خطای رایج در نسخه‌های اولیه گرامر برطرف شد.

۳.۳ سیستم انواع

مثال	نوع Python	نوع در DSL
128	int	int
1.0	float	float
false / true	bool	bool
"relu"	str	string
(64, 32, 32)	tuple[int]	shape
None	NoneType	None

تمایز بین `int` و `float` در زمان کامپایل بررسی می‌شود. مثلاً `Dropout(p=1)` خطا می‌دهد چون `p` انتظار `float` دارد نه `int`.

۴.۳ انواع لایه

۱.۴.۳ لایه‌های پارامتردار

این لایه‌ها در `__init__` به صورت `self.<id> = nn.X(...)` تولید می‌شوند.

کلیدواژه DSL	کلاس PyTorch	پارامترهای اصلی
Linear	<code>nn.Linear</code>	<code>bias, out_features, in_features</code>
Conv2d	<code>nn.Conv2d</code>	<code>padding, stride, kernel, out_ch, in_ch</code>
Conv1d	<code>nn.Conv1d</code>	<code>stride, kernel, out_ch, in_ch</code>
BatchNorm2d	<code>nn.BatchNorm2d</code>	<code>num_features</code>
LayerNorm	<code>nn.LayerNorm</code>	<code>normalized_shape</code>
MaxPool2d	<code>nn.MaxPool2d</code>	<code>stride, kernel</code>
AvgPool2d	<code>nn.AvgPool2d</code>	<code>stride, kernel</code>
Dropout	<code>nn.Dropout</code>	<code>p</code>
Flatten	<code>nn.Flatten</code>	<code>end_dim, start_dim</code>
Embedding	<code>nn.Embedding</code>	<code>embedding_dim, num_embeddings</code>
MultiHeadAttn	<code>nn.MultiheadAttention</code>	<code>num_heads, embed_dim</code>
LSTM	<code>nn.LSTM</code>	<code>num_layers, hidden_size, input_size</code>
GRU	<code>nn.GRU</code>	<code>hidden_size, input_size</code>

۲.۴.۳ فعال‌سازی‌ها

`ReLU`, `Sigmoid`, `Tanh`, `GELU`, `Softmax(dim)`, `LeakyReLU(negative_slope)`, `ELU(alpha)`.

۳.۴.۳ عملیات گراف (بدون ماژول)

این انواع در `__init__` خطی تولید نمی‌کنند و فقط در `forward` ظاهر می‌شوند:

کلیدواژه	پارامترها	کد تولیدشده
Add	—	<code>out = a + b</code>
Concat	<code>dim: int</code>	<code>out = torch.cat([a, b], dim=d)</code>
Residual	—	<code>out = main + shortcut</code>
Split	<code>chunks: int, dim: int</code>	<code>parts = torch.chunk(x, n, dim=d)</code>

۵.۳ نحو یال

تعریف ۱.۵.۳ > یال ساده

```
edge src -> dst
```

یک یال جهت‌دار از گره src به گره dst تعریف می‌کند.

تعریف ۲.۵.۳ > یال برچسب‌دار

```
edge src -> dst [label="name"]
```

یک یال با برچسب رشته‌ای تعریف می‌کند. برچسب‌ها دو نقش دارند:

- "shortcut": یال اتصال کوتاه به گره Residual() را مشخص می‌کند.
- "residual_*": اتصال کوتاه به گره‌ای با دو ورودی (الگوی Transformer).

۶.۳ کامنت‌ها

```
// □□□□ □□□□
/* □□□□
   □□□□ */
```

فصل ۴

پیاده‌سازی

۱.۴ معماری کامپایلر

تعریف ۱.۱.۴ > خط لوله کامپایل

کامپایلر چهار مرحله دارد:

۱. تجزیه (Parsing): `FileStream` → `NNGraphLexer` →
`NNGraphParser.program()` → درخت تجزیه
۲. تحلیل معنایی (Semantic Analysis): `ParseTreeWalker` +
`NNGraphCustomListener` → مدل گراف
۳. تولید کد (Code Generation): `CodeGenerator.generate()` → رشته Python
۴. خروجی: نوشتن به فایل `.py` یا چاپ در `stdout`

۱.۱.۴ مسئولیت فایل‌ها

فایل	نقش
NNGraph.g4	منبع حقیقت گرامر (۵۶ خط)
gen/NNGraphLexer.py	تولیدشده توسط ANTLR ۴ – نباید ویرایش شود
gen/NNGraphParser.py	تولیدشده توسط ANTLR ۴ – نباید ویرایش شود
gen/NNGraphListener.py	listener پایه با متدهای خالی – نباید ویرایش شود
custom_listener.py	تحلیل معنایی (۲۹۳ خط)
code_generator.py	تولید کد PyTorch (۲۳۹ خط)
main.py	رابط CLI و تابع compile_nng() (۷۳ خط)

۲.۴ تحلیل‌گر معنایی

تعریف ۱.۲.۴>NNGraphCustomListener

کلاس NNGraphCustomListener(NNGraphListener) در custom_listener.py تمام منطق تحلیل معنایی را پیاده می‌کند. حالت داخلی:

```
dict[id, {type, params, line}]:self.nodes •
list[{src, dst, label, line}]:self.edges •
dict[key, value]:self.config •
```

۱.۲.۴ انتشار مقدار در درخت

متدهای exitShape_expr و exitValue مقادیر را روی خود context ذخیره می‌کنند:

```
def exitShape_expr(self, ctx):
    ints = [int(ctx.INT_LITERAL(i).getText())
             for i in range(ctx.INT_LITERAL().__len__())]
    ctx.pvalue = tuple(ints)
    ctx.ptype = tuple
```

این رویکرد تضمین می‌کند که هر context پس از خروج از آن، مقادیر فرزندانش را در دسترس دارد – الگویی که در پروژه EVM مرجع نیز استفاده شده است.

۲.۲.۴ قوانین اعتبارسنجی

هشت قانون اعتبارسنجی در دو فاز اجرا می‌شوند:

فاز اول (در زمان پیمایش درخت):

۱. شناسه تکراری گره: در `exitNode_decl`
 ۲. ارجاع نامعتبر در یال: در `exitEdge_decl`
 ۳. نوع لایه ناشناس: در `exitNode_decl`
 ۴. عدم تطابق نوع پارامتر: در `exitNode_decl`
- فاز دوم (در `exitGraph_block`، پس از تکمیل گراف):
۵. گره خروجی اعلان‌نشده: در `_validate_output_declared`
 ۶. گره (هشدار): در `_validate_orphans`
 ۷. آریتی `Residual != ۲`: در `_validate_residual_arity`
 ۸. تشخیص دور با Kahn: در `_validate_no_cycles`
 ۹. قابلیت دسترسی BFS: در `_validate_reachability`

۳.۲.۴ قالب پیام خطا

قالب خطا

```
[line L:C] SemanticError: <message>
```

که L شماره سطر (از ۱) و C ستون توکن (از ۰) است. هشدارها فقط در `stderr` چاپ می‌شوند و کامپایل را متوقف نمی‌کنند.

۴.۲.۴ `_FakeCtx` برای خطاهای پس از پیمایش

خطاهایی که در `exitGraph_block` رخ می‌دهند context زنده ندارند. یک کلاس داخلی `_FakeCtx` شماره سطر را از رکورد اعلان گره حمل می‌کند:

```
class _FakeCtx:
    class start:
        line    = info["line"]
        column  = 0
```

۳.۴ مولد کد

تعریف ۱.۳.۴ > CodeGenerator

کلاس CodeGenerator در code_generator.py چهار گام داخلی دارد:

۱. `_compute_graph()`: ساخت لیست‌های مجاورت و مرتب‌سازی توپولوژیک
۲. `_assign_vars()`: تخصیص نام متغیر Python به هر گره
۳. `_emit_forward() + _emit_init()`: تولید خطوط کد
۴. `_assemble()`: ترکیب در سورس Python نهایی

۱.۳.۴ الگوریتم تخصیص متغیر

تعریف ۲.۳.۴ > اصل اساسی

`var_at[n] = 'x'` فقط وقتی که `out_degree[predecessor] == 1` وقتی یک گره به چند مقصد فرستاده می‌شود (`out_degree > 1`)، تنسور آن هنوز توسط شاخه‌های دیگر نیاز است. بازنویسی `x` در این نقطه آن را خراب می‌کند. به جای آن، گره بعدی نام شناسه خودش را به عنوان متغیر دریافت می‌کند.

چهار قانون کامل (به ترتیب در `_assign_vars`):

1. `var_at[input_id] = 'x'`
2. `(in_degree > 1):`
`var_at[n] = 'x' if out_degree[n] <= 1`
`var_at[n] = node_id if out_degree[n] > 1`
3. `(predecessor.out_degree > 1):`
`var_at[n] = node_id`
4. `(predecessor.out_degree == 1):`
`var_at[n] = var_at[predecessor]`

مثال ۳.۳.۴ > ردیابی MLP (زنجیره خطی)

$x \rightarrow \text{fc1} \rightarrow \text{relu1} \rightarrow \text{fc2} \rightarrow \text{relu2} \rightarrow \text{drop} \rightarrow \text{fc3} \rightarrow \text{out}$
 هر predecessor دارای $\text{out_degree} == 1$ است $\Rightarrow$ قانون ۴ همیشه اعمال می‌شود $\Rightarrow$ همه
 گره‌ها $\text{var_at} = 'x'$ دارند. کد forward از x در همه جا استفاده می‌کند.

مثال ۴.۳.۴ > ردیابی ResBlock (فن‌اوت سپس ادغام)

x به conv1 (مسیر اصلی) و skip (اتصال کوتاه) فرستاده می‌شود. پس $\text{out_degree}[x] = 2$
 $\text{var_at} = 'conv1' \Rightarrow$ قانون ۳
 $\text{'conv1'} \Rightarrow$ همه $\text{bn2, conv2, relu1, bn1}$: قانون ۴
 $\text{var_at} = 'x' \Rightarrow \text{out_degree}=1, 2$: قانون ۲ (Residual, skip)
 کد تولیدشده: $x = \text{conv1} + x$

مثال ۵.۳.۴ > ردیابی InceptionBlock (دو شاخه موازی)

$\text{out_degree}[x] = 2$ (به convB1 و convA1)
 $\text{'convA1'} \Rightarrow$ قانون ۳
 $\text{'convB1'} \Rightarrow$ قانون ۳
 $\text{'convA1'} \Rightarrow$ ارث از قانون ۴
 $\text{'convB1'} \Rightarrow$ ارث از قانون ۴
 $\text{'x'} \Rightarrow$ قانون ۲ (Concat, cat1)
 کد: $x = \text{torch.cat}([\text{convA1}, \text{convB1}], \text{dim}=1)$

مثال ۶.۳.۴ > ردیابی TransformerEncoder (دو زیرلایه residual ضمنی)

$\text{out_degree}[x] = 2$ (به norm1 و attn)
 $\text{out_degree}[\text{norm1}] = 2$ (به norm2 و ff1)
 $\text{'attn'} \Rightarrow$ قانون ۳
 $\text{'attn'} \Rightarrow$ قانون ۴
 $\text{'norm1'} \Rightarrow$ قانون ۲ (in_degree=2, out_degree=2)
 $\text{'ff1'} \Rightarrow$ قانون ۳
 $\text{'ff1'} \Rightarrow$ قانون ۴ (gelu1, drop2, ff2)
 $\text{'x'} \Rightarrow$ قانون ۲ (in_degree=2, out_degree=1)
 خطوط تولیدشده:
 $\text{attn, _} = \text{self.attn}(x, x, x)$

```

attn      = self.drop1(attn)
norm1     = self.norm1(x + attn)
ff1       = self.ff1(norm1)
ff1       = self.gelu1(ff1)
ff1       = self.drop2(ff1)
ff1       = self.ff2(ff1)
x         = self.norm2(norm1 + ff1)

```

۲.۳.۴ موارد خاص تولید کد

الگو	شرط	کد تولیدشده
MultiHeadAttn	نوع لایه	<code>out, _ = self.attn(x, x, x)</code>
LSTM / GRU	نوع لایه	<code>out, _ = self.lstm(x)</code>
Residual	گره بدون ماژول	<code>out = main + shortcut</code>
Add	گره بدون ماژول	<code>out = a + b</code>
Concat	گره بدون ماژول	<code>out = torch.cat([a, b], dim=d)</code>
Split	گره بدون ماژول	<code>parts = torch.chunk(x, n, dim=d)</code>
Residual ضمنی	<code>in_degree > 1</code> + برچسب <code>* residual_</code>	<code>out = self.norm(skip + main)</code>

نگاشت نام پارامتر: DSL از نام‌های کوتاه‌تر استفاده می‌کند. مولد کد آن‌ها را به نام‌های PyTorch نگاشت می‌کند: `in_ch → in_channels`، `out_ch → out_channels`، `kernel_size`. همه نام‌های دیگر بدون تغییر عبور می‌کنند.

۳.۳.۴ MultiHeadAttn در batch_first

`nn.MultiheadAttention` از PyTorch به صورت پیش‌فرض `batch_first=False` است. مولد کد همیشه `batch_first=True` اضافه می‌کند تا با شکل رایج `(batch, seq, dim)` سازگار باشد.

۴.۳.۴ ساختار کد خروجی

```

import torch
import torch.nn as nn

```

```
class <ModelName>(nn.Module):
    def __init__(self):
        super(<ModelName>, self).__init__()
        <init_lines>

    def forward(self, x):
        <forward_lines>
        return <output_var>

if __name__ == '__main__':
    device = torch.device('<device>')
    model = <ModelName>().to(device)
    x = torch.randn(<batch_size>, <input_shape...>).to(device)
    print(model(x).shape)
```

فصل ۵

ارزیابی

۱.۵ مجموعه آزمایش

تعریف ۱.۱.۵ • آمار آزمایش

- تعداد کل تست‌ها: ۵۵
- فریم‌ورک: pytest نسخه ۲.۴.۸
- زمان اجرا: حدود ۹.۱ ثانیه

فایل	تست‌ها	پوشش
test_parser.py	۸	گرامر بدون خطای نحوی برای ۴ نمونه؛ config، پرچسب، شکل
test_listener.py	۱۴	۴ ورودی معتبر؛ ۱۰ شرط خطا با SystemExit
test_codegen.py	۱۹	نام کلاس‌های nn، مقادیر پارامتر، الگوهای forward
test_e2e.py	۱۴	کامپایل کامل + ایجاد نمونه + for- pass ward با شکل صحیح

۲.۵ نمونه‌های کارکرده

مثال ۱.۲.۵ > MLP – زنجیره خطی ۳ لایه

- ورودی: $\text{tensor}(784)$ – خروجی: $(\text{batch}, 10)$
- توپولوژی: زنجیره خطی
- استراتژی متغیر: همه x
- آزمایش: Softmax ردیف‌ها به ۱ جمع می‌شوند ($\text{atol}=1e-5$)

مثال ۲.۲.۵ > ResBlock – بلوک کانولوشنی با اتصال باقیمانده

- ورودی: $\text{tensor}(64, 32, 32)$ – خروجی: $(\text{batch}, 65536)$ پس از Flatten
- توپولوژی: فن‌اوت از ورودی، ادغام در $\text{Residual}()$
- آزمایش: شکل صحیح؛ در حالت eval خروجی قطعی است

مثال ۳.۲.۵ > InceptionBlock – دو شاخه Conv موازی

- ورودی: $\text{tensor}(32, 56, 56)$ – خروجی: $(\text{batch}, 48, 56, 56)$
- شاخه A: ۱۶ کانال؛ شاخه B: ۳۲ کانال؛ ادغام: $\text{Concat}(\text{dim}=1)$ کانال $48 \rightarrow$
- آزمایش: ابعاد فضایی تغییر نمی‌کنند

مثال ۴.۲.۵ > TransformerEncoder – یک لایه encoder

- ورودی: $\text{tensor}(512, 128)$ – خروجی: $(1024, 128)$ پس از Flatten
- دو زیرلایه residual ضمنی (بدون گره $\text{Residual}()$)
- MultiHeadAttn با $\text{embed_dim}=128$, $\text{num_heads}=8$
- آزمایش: شکل صحیح؛ در حالت eval خروجی قطعی است

۳.۵ اثبات درستی متغیرها

تعریف ۱.۳.۵ > اثبات عدم بازنویسی تنسور

- برای هر نمونه شاخه‌دار، $\text{data_ptr}()$ تنسور ورودی در طول محاسبات forward ردیابی شد:
- **ResBlock**: $\text{data_ptr}()$ در نقطه $x = \text{conv1} + x$ تغییر نکرده
 - **Inception**: $\text{data_ptr}()$ هنگام فراخوانی $\text{convA1}(x)$ و $\text{convB1}(x)$ یکسان

است

• **Transformer**: `x.data_ptr()` هنگام `norm1(x + attn)`
`norm1.data_ptr()` هنگام `norm2(norm1 + ff1)` تغییر نکرده

این نتیجه مستقیماً از اصل `var_at[n]='x'` فقط وقتی `out_degree=1` پیروی می‌کند.

۴.۵ آمار کدبیس

خطوط	فایل/دایرکتوری
۵۶	NNGraph.g4
۲۹۳	custom_listener.py
۲۳۹	code_generator.py
۷۳	main.py
۵۷۵	tests/ (۴ فایل)
۱۰۰≈	required_code_collection/
۱۳۳۶≈	جمع

- زبان: Python ۳.۱۴.۳
- ANTLR: ۲.۱۳.۴
- PyTorch: ۰.۱۰.۲
- pytest: ۲.۴.۸

فصل ۶

تصمیمات طراحی

۱.۶ Listener در برابر Visitor

الگوی Listener به صورت طبیعی با انباشت حالت سازگار است. متدهای `exit*` می‌توانند مستقیماً روی `self.nodes` و `self.edges` عمل کنند بدون اینکه مقادیر را از طریق درخت برگردانند. برای یک کامپایلر گراف که هدفش ساختن یک مدل گراف از درخت تجزیه است، این انتخاب طبیعی‌تر از Visitor است.

۲.۶ `load_from_listener()` در برابر `generate(traversal)`

پروژه EVM مرجع از `CodeGenerator.generate(traversal)` استفاده می‌کند که یک لیست توکن پس‌ترتیب دریافت می‌کند—مناسب برای تولید بایت‌کد پشت‌های. برای کامپایلر گراف، ورودی طبیعی ساختار گراف (گره‌ها + یال‌ها) است نه جریان خطی توکن. پس `load_from_listener(listener)` مستقیم‌تر است.

۳.۶ نام‌گذاری متغیر با `node_id`

مثال‌های مشخصات زبان از نام‌های تک‌حرفی `(a, b)` یا نام‌های توصیفی `(attn_out, identity)` استفاده می‌کنند. این کامپایلر از شناسه واقعی گره `(attn, conv1, convA1)` استفاده می‌کند زیرا:

- قطعی است
- خوانا است
- برای گراف‌های بزرگ‌تر بدون شمارنده کار می‌کند
- خروجی ریاضی یکسان است

فصل ۷

نتیجه‌گیری و کارهای آینده

۱.۷ نتیجه‌گیری

NNGraph DSL یک زنجیره کامپایلر چهار مرحله‌ای از `nng` به `nn.Module` قابل اجرا ارائه می‌دهد. کامپایلر:

- گرامر FANTLR با ۱۴ قانون تجزیه‌گر و ۲۰ توکن
- ۲۴ نوع لایه را پشتیبانی می‌کند
- ۸ قانون اعتبارسنجی را اجرا می‌کند
- شاخه‌بندی پیچیده را بدون بازنویسی تنسور مدیریت می‌کند
- با ۵۵ تست در چهار معماری نمونه اعتبارسنجی شده است

۲.۷ کارهای آینده

این موارد در مشخصات زبان پیش‌بینی شده‌اند اما پیاده‌سازی نشده‌اند:

قابلیت	توضیح
استنتاج شکل	انتشار شکل تنسور در طول گراف و تشخیص عدم تطابق در کامپایل
صادرکردن ONNX	تولید گراف سازگار با ONNX به جای Python
داربست آموزش	تولید حلقه آموزش با بهینه‌ساز و تابع خطا از بلوک config
تجسم Graphviz	خروجی نمودار گراف مدل
زیرگراف‌های نام‌دار	بلوک‌های ماکرو قابل استفاده مجدد (مثلاً ResBlock قابل تکرار)
کنترل جریان پویا	شاخه‌های else/if درون forward
حاشیه‌نویسی کوانتیزاسیون	متادیتا برای تبدیل مدل به نسخه‌های کوانتیزه

منابع

۱. Parr, T. (۲۰۱۳.) The Definitive ANTLR ۴ Reference. Pragmatic Bookshelf.
۲. Paszke, A., et al. (۲۰۱۹.) PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS ۲۰۱۹.
۳. NADER Framework: Neural Architecture Design via Representation (referenced in NNGraph_DSL_Specification.md.)
- Zhang, A. M. amznotes LaTeX template. github.com/alexmingzhang/latex-notes-template